PROJECT TITLE:

BIG DATA - EARTHSCAPE-CLIMATE-AGENCY

Developer Guide

Name	StudentID
Aqsa Kamran	Student1392300
Muskan Kamran	Student1384349
Ifrah Khanm	Student1390824
Linta Shehzad	Student1392273

Faculty:

Sir Muhammad Usman

```
reply = f"🌧 {city_cleaned} ka forecast: Aane wale waqt mein temperatur
{temp}°C hoga, weather: {desc}."
return jsonify({ "reply": reply })
:
return jsonify({ "reply": f"⚠ {city_cleaned} ka forecast data mil nahi
" })

default to current
= f"http://api.openweathermap.org/data/2.5/weather?q={city_cleaned}&appi
NWEATHER_API_KEY}&units=metric"
= requests.get(url, timeout=5)
raise_for_status()
her_data = res.json()

eather_data.get("cod") == 200:
temp = weather_data["main"]["temp"]
desc = weather_data["weather"][0]["description"].capitalize()
humidity = weather_data["main"]["humidity"]

reply = f"📍 {city_cleaned} mein temperature abhi {temp}°C hai, weather
{desc}, aur humidity {humidity}% hai."
return jsonify({ "reply": reply })
```

```
Main Content -->
class="main">
iv class="main-content-wrapper">
<div class="admin-header d-flex justify-con
  <div class="d-flex align-items-center gap
    <img src="{{ user.profile_img or '/stat
    width="40"
      height="40" style="object-fit: cover;
    <h5 class="mb-0">{{ user.username or 'A
  </div>
  <div class="text-end">
    <i class="fas fa-circle-user me-1"></i>
    <span class="badge bg-success ms-2">{{
  </div>
</div>

<div id="datetime" class="text-muted mb-3">

<div class="alert alert-info alert-dismissi
  Tip: Check for climate anomalies in th
  <button type="button" class="btn-close" d
</div>

<div class="row g-4 mb-4">
TPUT  DEBUG CONSOLE  TERMINAL  PORTS
```

Dashboard
Upload Data
Visualize
Model
Alerts
Profile
Feedback

Table of Contents

1. Project Overview 3
2. Technology Stack 3
3. Project Structure 4
4. Dataset & Model Training 4
5. AI Chatbot (EarthScape Bot) 5
6. Weather API Integration 5
7. System Setup & Installation 6
8. ML Models & Joblib Files 7
9. SMTP Email Alerts 7
10. MongoDB Connection 7
11. Role-Based Dashboards (Analyst & Admin) 9
12. Code Snippets: 10
Final Summary 23

1. Project Overview

- EarthScape Climate Agency is a web-based application designed to allow users to upload, visualize, and predict climate data using machine learning models.
- It provides personalized climate predictions based on user data.
- The system integrates real-time weather information.
- Automatic alert notifications are sent to users when critical conditions arise.
- An AI-powered chatbot is available on every page to assist users with queries and support.

2. Technology Stack

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Python with Flask framework
- **Database:** MongoDB (primary), SQLite (users.db) for lightweight user data storage
- **Machine Learning Libraries:** scikit-learn, pandas, numpy
- **Visualization:** matplotlib, seaborn
- **External APIs:**
  - OpenWeatherMap API for live weather data
  - Gemini API for AI chatbot integration
- **Email Service:** SMTP (using Flask-Mail with Gmail SMTP) for sending alerts

3. Project Structure

- climate_dashboard/venv/ – Python virtual environment containing dependencies
- static/js/ – JavaScript files
- static/uploads/ – Temporary storage for uploaded CSV files
- static/illustration.jpeg, illustration.jpg, logo.png – Image assets for UI
- templates/admin/ – Admin-only HTML pages
- templates/analyst/ – Analyst dashboard UI pages
- templates/layout/ – Common layout templates (navbar, footer, etc.)
- templates/login.html – User login page
- templates/register.html – User registration form
- app.py – Main Flask application file handling routing and logic
- humidity_model.joblib, temperature_model.joblib – Pre-trained ML models
- users.db – SQLite file for user data
- requirements.txt – List of Python dependencies

4. Dataset & Model Training

- The system uses structured CSV datasets containing temperature, humidity, rainfall, and windspeed data.

- Models are trained using scikit-learn with algorithms including:
  - Linear Regression
  - Decision Tree
  - Random Forest
  - Support Vector Regression (SVR)
  - Polynomial Regression
- Trained models are saved as .joblib files and dynamically loaded during predictions.

5. AI Chatbot (EarthScape Bot)

- Built with Google Gemini API.
- Accessible as a floating chatbot on every page.
- Handles natural language queries related to weather, system usage, alerts, and predictions.
- Trained on prompts related to the application's functionality.

6. Weather API Integration

- Uses OpenWeatherMap API to fetch live weather data based on user location and input.
- Retrieves real-time values for temperature, humidity, rainfall, and windspeed.
- Weather data is also integrated into chatbot responses.

7. System Setup & Installation

Prerequisites:

- Python 3.10 or higher
- MongoDB installed locally
- Internet connection for API access

Setup Steps:

1. Create a virtual environment: python -m venv venv
2. Activate the virtual environment:
    - Windows: venv\Scripts\activate
    - macOS/Linux: source venv/bin/activate
3. Install required packages: pip install -r requirements.txt
4. Run the application: python app.py

8. ML Models & Joblib Files

- Models are trained using scikit-learn and saved as .joblib files.
- Models load dynamically depending on the user's prediction choice (rainfall, temperature, or humidity).
- Optional input like windspeed is supported to improve prediction accuracy.
- Predictions are generated in real-time and logged into the database.

9. SMTP Email Alerts

- Flask-Mail is configured with Gmail SMTP for sending real-time email alerts.
- Alerts are triggered when predictions exceed safety thresholds or severe anomalies are detected.
- SMTP settings (mail server, port, sender email, and app password) are configured in app.py.
- Alerts appear both as on-screen notifications and email messages.
- All alert events are logged in the database.

10. MongoDB Connection

- Database name: climate_data
- Collections include:

- users: stores user credentials, roles, and profile images
- predictions: stores user inputs, selected models, predicted results, and timestamps
- alerts: records alert type, value, severity, user ID, and send status
- feedback: stores user-submitted feedback messages

```
# 2. For MONGODB ATLAS (Cloud Database):
#    Go to your MongoDB Atlas dashboard -> "Connect" -> "Connect your application".
#    Copy the provided connection string. It will look similar to this:
#    "mongodb+srv://<username>:<password>@your-cluster-name.abcde.mongodb.net/earthscape_db?retryWrites=true&w=majority"
#    Replace <username>, <password>, and 'your-cluster-name'.
#    Also, ensure you have configured Network Access in Atlas to allow your IP address.
app.config["MONGO_URI"] = "mongodb+srv://<username>:<password>@cluster0.vedoeab.mongodb.net/earthscape_db?retryWrites=true&w=majority&
appName=Cluster0"
# Set a secret key for session management and flash messages.
# IMPORTANT: Change this in production to a strong, randomly generated string!
app.secret_key = 'your_super_secure_and_random_secret_key_for_production'

# Initialize PyMongoSS
mongo = PyMongo(app)
```

11. Role-Based Dashboards (Analyst & Admin)

- The system includes two distinct dashboards designed for different user roles: Analyst and Admin.

Analyst Dashboard:

- Provides a personalized interface for analysts to interact with climate data.
- Allows uploading of CSV files for data input.
- Enables making predictions using various machine learning models.
- Displays prediction history and logs for user reference.
- Offers dynamic visualizations to help analyze climate trends.
- Focused on user-level forecasting and detailed data exploration.

Admin Dashboard:

- Provides system-level controls and monitoring tools.
- Displays metrics such as total users, total predictions made, and alerts triggered.
- Allows admins to review feedback submitted by users.
- Gives access to complete prediction histories across the platform.
- Designed to manage overall platform health and ensure data integrity.

12. Code Snippets:

- Imports and main configuration:

```
# Import necessary Flask modules and MongoDB extensions
from flask import Flask, json, request,Blueprint, render_template, redirect, url_for, jsonify, flash, session
from flask_pymongo import PyMongo
import pandas as pd
import csv , io
import google.generativeai as genai
from pymongo import MongoClient
from bson.objectid import ObjectId
from werkzeug.security import generate_password_hash, check_password_hash # For secure password handling
from bson.objectid import ObjectId # To work with MongoDB's default _id field (optional but good practice)
import datetime
from flask_mail import Mail, Message # For sending emails (optional, if you want to implement email notifications)
import re


# Initialize the Flask application
app = Flask(__name__)

visualize_bp = Blueprint('visualize', __name__)

# ✅ Mail setup (put in main file once globally)
app.config['MAIL_SERVER'] = 'smtp.gmail.com'
app.config['MAIL_PORT'] = 587
app.config['MAIL_USE_TLS'] = True
app.config['MAIL_USERNAME'] = 'muskankamran3369@gmail.com'         # 🔁 Replace with your email
app.config['MAIL_PASSWORD'] = 'fzzb khaj jgom ybdr'       # 🔁 App password, not normal one
app.config['MAIL_DEFAULT_SENDER'] = 'muskankamran3369@gmail.com'
mail = Mail(app)
```

- EarthScape Chatbot:

```python
@app.route("/chatbot", methods=["POST"])
def chatbot():
    user_msg = request.form.get("message", "").strip()

    try:
        model = genai.GenerativeModel("gemini-2.0-flash")

        # Step 1: Check if message is weather-related
        weather_check_prompt = f"""
The user will input a message. You have to say only "Yes" or "No" based on
whether the message is related to weather, climate, temperature, humidity, rain, or environmental conditions.

Message: "{user_msg}"
Reply with only Yes or No.
"""
        weather_check = model.generate_content(weather_check_prompt)
        is_weather_related = weather_check.text.strip().lower() == "yes"

        # Step 2: If weather-related, extract city
        if is_weather_related:
            location_prompt = f"""
Extract the city or location name from this message. If none found, say "None".

Message: "{user_msg}"
City:
"""
            city_response = model.generate_content(location_prompt)
            city_raw = city_response.text.strip()
            city_cleaned = re.sub(r"[^a-zA-Z\s]", "", city_raw).strip().title()

            if city_cleaned and city_cleaned.lower() != "none":
```

- Registration:

```python
# --- Registration Route ---
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        full_name = request.form.get('name')
        email = request.form.get('email')
        password = request.form.get('password')
        role = "analyst"  # ✅ Force default role to Analyst

        if not all([full_name, email, password]):
            flash('All fields are required!', 'danger')
            return render_template('register.html')

        existing_user = users_collection.find_one({'email': email})
        if existing_user:
            flash('This email address is already registered. Please try logging in or use a different email.', 'danger')
            return render_template('register.html')

        hashed_password = generate_password_hash(password)
        user_data = {
            'full_name': full_name,
            'email': email,
            'password_hash': hashed_password,
            'role': role  # Always Analyst
        }
```

- Login:

```python
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form.get('email')
        password = request.form.get('password')

        user_data = users_collection.find_one({'email': email})

        if user_data and check_password_hash(user_data['password_hash'], password):
            session['user_id'] = str(user_data['_id'])
            session['username'] = user_data['full_name']
            session['user_role'] = user_data['role']

            # Redirect based on role
            if user_data['role'] == 'admin':
                return redirect(url_for('admin_dashboard'))
            elif user_data['role'] == 'analyst':
                return redirect(url_for('analyst_dashboard'))
            else:
                flash("Access denied: Unknown role", "danger")
                return redirect(url_for('login'))
        else:
            flash("Invalid email or password", "danger")
            return render_template('login.html')

    return render_template('login.html')
```

- Admin dashboard:

```python
@app.route('/admin')
def admin_dashboard():
    user_id = session.get('user_id')
    user = mongo.db.users.find_one({'_id': ObjectId(user_id)})
    return render_template('layout/admin_dashboard.html', user=user)

@app.route('/admin/home')
def admin_home():
    return render_template('admin/admin_home.html')
```

- Admin managing users:

```python
@app.route('/admin/users')
def admin_users():
    users = list(mongo.db.users.find().sort('full_name', 1))
    return render_template('admin/admin_users.html', users=users)

@app.route('/admin/users/delete/<id>', methods=['POST'])
def delete_user(id):
    mongo.db.users.delete_one({'_id': ObjectId(id)})
    return '', 204

@app.route('/admin/users/edit/<id>', methods=['POST'])
def edit_user(id):
    data = request.get_json()
    mongo.db.users.update_one(
        {'_id': ObjectId(id)},
        {'$set': {
            'full_name': data['full_name'],
            'email': data['email'],
            'role': data['role']
        }}
    )
    return '', 204
```

- Admin models evaluation:

```python
@app.route('/admin/evaluate-models')
def evaluate_models():
    try:
        target = request.args.get("target")
        df = pd.DataFrame(list(mongo.db.climate_records.find()))
        df = df[['temperature', 'humidity', 'rainfall', 'wind_speed']].dropna()

        # Feature selection
        if target == "humidity":
            features = ["temperature"]
        elif target == "rainfall":
            features = ["temperature", "humidity"]
        elif target == "temperature":
            features = ["humidity", "rainfall"]
        else:
            return jsonify({"success": False, "error": "Invalid target"})

        df = df.dropna(subset=features + [target])
        X = df[features]
        y = df[target]

        models = {
            "Decision Tree": DecisionTreeRegressor(),
            "Linear Regression": LinearRegression(),
            "Random Forest": RandomForestRegressor(),
            "SVR": SVR(),
            "Polynomial": make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
        }
```

- Analyst dashboard:

```python
@app.route('/analyst')
def analyst_dashboard():
    user_id = session.get('user_id')
    user = mongo.db.users.find_one({'_id': ObjectId(user_id)})

    total_records = mongo.db.climate_records.count_documents({})
    total_alerts = mongo.db.climate_alerts.count_documents({})
    predictions_count = mongo.db.climate_records.count_documents({'source': 'Model + Manual'})

    # ✅ Fetch recent activity
    username = session.get('username', 'unknown')
    recent_activities = list(mongo.db.activities.find(
        {"username": username}
    ).sort("timestamp", -1).limit(5))

    return render_template('layout/analyst_dashboard.html', user=user,
                           stats={
                               "records": total_records,
                               "alerts": total_alerts,
                               "predictions": predictions_count,
                               "visualizations": 0
                           },
                           recent_activities=recent_activities)


@app.route("/analyst/home")
def analyst_dashboard_home():
    user_id = session.get('user_id')
    user = mongo.db.users.find_one({'_id': ObjectId(user_id)})
```

- Analyst upload scv file:

```python
# ✅ Improved CSV Upload Route with Validation, Alerts, and Prediction Options
@app.route("/analyst/upload", methods=["GET", "POST"])
def upload_climate_records_analyst():
    if request.method == 'POST':
        if "file" not in request.files:
            return jsonify({"success": False, "message": "No file uploaded."})

        file = request.files["file"]
        filename = file.filename.lower()

        if filename.endswith('.csv'):
            try:
                stream = io.StringIO(file.stream.read().decode("UTF8"))
                reader = csv.DictReader(stream)
                data = list(reader)

                required_fields = ['temperature', 'humidity', 'rainfall', 'wind_speed']
                inserted = 0
                skipped = 0
                errors = []
                alerts = []

                for i, row in enumerate(data):
                    if not all(field in row and row[field].strip() for field in required_fields):
                        errors.append(f"Row {i+2}: Missing fields.")
                        skipped += 1
                        continue

                    try:
                        temp = float(row["temperature"])
```

- Model training:

```python
@app.route('/analyst/train-model', methods=['POST'])
def train_model_analyst():
    try:
        target = request.form.get("target")
        model_type = request.form.get("ml_model")
        df = pd.DataFrame(list(mongo.db.climate_records.find()))
        df = df[['temperature', 'humidity', 'rainfall', 'wind_speed']].dropna()

        # ✅ Select features based on target
        if target == "humidity":
            features = ["temperature"]
        elif target == "rainfall":
            features = ["temperature", "humidity"]
        elif target == "temperature":
            features = ["humidity", "rainfall"]
        else:
            return jsonify({"success": False, "error": "Invalid target"})

        df = df.dropna(subset=features + [target])

        # ✅ Model selection
        if model_type == "decision_tree":
            model = DecisionTreeRegressor()
        elif model_type == "linear_regression":
            model = LinearRegression()
        elif model_type == "svr":
            model = SVR(kernel='rbf')
        elif model_type == "polynomial":
            model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
        elif model_type == "Random":
```

- Email alert via SMTP:

```python
# ✅ Send alert email
user = mongo.db.users.find_one({'username': session.get('username')})
if user and 'email' in user:
    try:
        subject_line = "📢 EarthScape Climate Agency - Climate Alert Notification"
        message_body = f"""
        Dear {user['username']},

        We hope you're doing well.

        This is an automated alert from EarthScape Climate Agency regarding unusual
        climate conditions observed during your recent prediction request:

        {chr(10).join(f"- {msg}" for msg in alert_messages)}

        Please take necessary precautions if you're in the affected area.

        Stay safe and thank you for using EarthScape Climate Agency.

        Best regards,
        🌿 EarthScape Climate Team
        """

        msg = Message(
            subject=subject_line,
            sender=app.config['MAIL_USERNAME'],
            recipients=[user['email']],
            body=message_body
        )
        mail.send(msg)
```

- Visualizations:

```python
@app.route('/analyst/visualize')
def analyst_visualize():
    return render_template('analyst/analyst_visualize.html')
@app.route('/analyst/model')
def analyst_model():
    return render_template('analyst/analyst_model.html')

@app.route('/api/climate-data')
def get_climate_records():


    data = list(mongo.db.climate_records.find({}, {'_id': 0, 'timestamp': 1, 'temperature': 1, 'rainfall': 1}))

    formatted = []
    for d in data:
        ts = d.get('timestamp')

        date_str = "unknown"
        try:
            if isinstance(ts, datetime):
                date_str = ts.strftime('%Y-%m-%d')
            elif isinstance(ts, str):
                date_str = datetime.fromisoformat(ts).strftime('%Y-%m-%d')
            elif isinstance(ts, int):  # UNIX timestamp
                date_str = datetime.fromtimestamp(ts).strftime('%Y-%m-%d')
        except:
            pass

        formatted.append({
            'date': date_str,
```

\

- Alerts & feedback:

```python
@app.route('/analyst/get-alerts')
def get_alerts():
    try:
        alerts = list(mongo.db.climate_alerts.find().sort("timestamp", -1))
        for a in alerts:
            a["_id"] = str(a["_id"])
            a["timestamp"] = a["timestamp"].isoformat()
        return jsonify({"success": True, "alerts": alerts})
    except Exception as e:
        return jsonify({"success": False, "error": str(e)})



@app.route('/analyst/alerts')
def analyst_alerts():
    return render_template('analyst/analyst_alerts.html')



@app.route('/analyst/submit_feedback', methods=['POST'])
def submit_feedback_ajax():
    message = request.form.get('message')
    username = session.get('username', 'unknown')
    mongo.db.feedback.insert_one({
        'username': username,
        'message': message,
        'timestamp': datetime.datetime.now()
    })
    # ✅ Log feedback
    mongo.db.activities.insert_one({
        "username": username,
```

- Prediction history:

```python
@app.route('/analyst/get-prediction-history')
def get_prediction_history():
    try:
        username = session.get("username", "unknown")
        history = list(mongo.db.prediction_history.find({"username": username}).sort("timestamp", -1))

        for record in history:
            record["_id"] = str(record["_id"])  # convert ObjectId to string
            record["timestamp"] = record["timestamp"].isoformat()  # make datetime JSON serializable

        return jsonify({"success": True, "history": history})

    except Exception as e:
        return jsonify({"success": False, "error": str(e)})
```

- Main App run:

```python
# --- Run the Flask application ---
if __name__ == '__main__':
    # Set debug to True for development. Disable in production.
    app.run(debug=True)
```

Final Summary

EarthScape Climate Agency is a comprehensive web-based platform that empowers users to upload, visualize, and predict climate data using advanced machine learning techniques. By integrating real-time weather information and offering personalized predictions, the system enables proactive climate monitoring and decision-making.

The application supports multiple user roles through dedicated Analyst and Admin dashboards, facilitating both detailed data analysis and system management. An AI-powered chatbot enhances user experience by providing instant support and guidance, while automated email alerts ensure timely notification of critical climate events.

Built with a robust technology stack—including Python Flask backend, MongoDB, and scikit-learn ML models—EarthScape Climate Agency exemplifies a scalable, user-centric solution for climate data analysis and forecasting.